

94. Le petit jeu : tout ensemble

Variables, conditions, boucles, tableaux, fonctions, struct : un vrai jeu, et rien d'autre.

1. Ce qu'il faut comprendre

Des objets tombent du ciel ; le panier (`|#|`) les attrape. Chaque objet attrapé rapporte un point ; chaque objet manqué coûte une vie. Toutes les notions du cours y sont, **à leur place** :

Variables : `score`, `vies`, la position du panier. **Conditions** : attrapé ou manqué ? partie finie ? **Boucles** : la boucle du jeu, et un `for` sur les objets. **Tableau de struct** : les trois `Chute`. **Fonctions** : `lancer`, `attrape`, `dessinerPanier`, `dessinerScore`. **Méthode** : `tomber()`. **VBlank** : le décor posé écran éteint, et chaque image n'écrit que ce qui a changé.

Un objet n'est jamais créé ni détruit : il est **recyclé**. Quand il arrive en bas, `lancer(i)` le renvoie en haut, à une colonne tirée au hasard. C'est la technique de presque tous les jeux Game Boy : **un nombre fixe de places**, qu'on réutilise.

`attrape(i)` est une fonction qui **répond** : l'objet est-il sur la ligne du panier, **et** entre ses deux bords ? Sa condition se lit comme la règle du jeu.

Le panier se redessine **seulement s'il a bougé**, le score **seulement quand il change**, et les objets **un tour sur huit** : c'est ce qui permet au panier de rester vif pendant que la pluie tombe.

Quand `vies` tombe à 0, on sort de la boucle du jeu par `break`, et le programme continue : « PERDU! ». Le `while (true)` de la fin garde l'écran affiché.

À toi : accélère la chute quand le score augmente (indice : remplace le 8 de `attente == 8` par une variable qui diminue).

Les fonctions de cette leçon — et le tuto de chacune

- `#include <hasard>` → [0.81.2. La fonction hasard\(\) — un nombre au hasard](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <ecran>` → [27.1. La fonction ecran\(\) — éteindre et rallumer l'écran](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1452 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          73 o ← la console, à l'allumage — la mise en route de la console
Chute::tomber()     20 o ← main() — ta fonction
lancer()            42 o ← main() — ta fonction
```

```

attrape()          62 o ← main() – ta fonction
dessinerPanier()   41 o ← main() – ta fonction
dessinerScore()    21 o ← main() – ta fonction
main()             410 o ← démarrage – ta fonction
VBlank             8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage      28 o ← image() – attend l'image suivante
AttendreVBlank     16 o ← ecran() – attend que l'écran soit libre
AttendreAcces      12 o ← EcrireTexte, EcrireNombre – attend de pouvoir écrire une case
AdresseCase        10 o ← texte() – trouve l'adresse d'une case de l'écran
EcrireTexte        10 o ← texte() – écrit un texte, lettre par lettre
EcrireNombre       33 o ← nombre() – écrit un nombre en chiffres
Reste              10 o ← lancer() – le reste d'une division
CopierTuiles       85 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte       14 o ← démarrage – vide les deux cartes de l'écran
Hasard             25 o ← hasard() – tire un nombre au hasard
LireManette        37 o ← bouton() – lit les huit boutons
Tuiles             448 o ← CopierTuiles – 28 lettres sur 43
" "               3 o ← texte() – un texte gravé
"|#|"             3 o ← texte() – un texte gravé
"SCORE:"          6 o ← texte() – un texte gravé
"VIES:"           5 o ← texte() – un texte gravé
"GAUCHE DROITE"   16 o ← texte() – un texte gravé
" "              1 o ← texte() – un texte gravé
"0"              1 o ← texte() – un texte gravé
"PERDU!"          6 o ← texte() – un texte gravé
"SCORE:"          6 o ← texte() – un texte gravé
Lettres gravées : ACDEGHIOPRSTUV0123456789!:#|
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, EffacerFond, EffacerPanneau, DonneesTransfert, Notes
Fonctions du langage appelées :
    texte()          x 9   171 o sur place, dans dessinerPanier(), main() + AdresseCase (10 o), EcrireTexte (10 o)
    nombre()         x 3    30 o sur place, dans dessinerScore(), main() + EcrireNombre (33 o)
    ecran()          x 2    21 o sur place, dans main() + AttendreVBlank (16 o)
    bouton()         x 2    10 o sur place, dans main() + LireManette (37 o)
    image()          x 2     6 o sur place, dans main() + AttendreImage (28 o)
    hasard()         x 1     3 o sur place, dans lancer() + Hasard (25 o)
Tes fonctions appelées :
    lancer()         x 3    21 o sur place, dans main()
    dessinerScore() x 3     9 o sur place, dans main()
    attrape()        x 1     7 o sur place, dans main()
    dessinerPanier() x 2     6 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : LE PETIT JEU. Des gouttes 0 tombent ; un panier|#| en bas
// les attrape. Chaque goutte attrapée = 1 point ; chaque goutte ratée = 1 vie en moins.
// À 0 vie : PERDU!
//
// Tout ce qu'on a vu sert ici : variables, conditions, boucles, tableaux,
// fonctions et struct.

#include <hasard> // tire un nombre au hasard
#include <texte>   // écrit un texte à l'écran
#include <nombre> // écrit un nombre en chiffres
#include <ecran>  // éteint ou rallume l'écran
#include <bouton> // lit un bouton de la manette
#include <reste>  // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair

```

2. UNE CONSTANTE — COMBIEN

```
const uint8_t COMBIEN = 3; // 3 gouttes à la fois.
```

3. UNE CONSTANTE — SOL

```
const uint8_t SOL = 16; // La ligne du panier.
```

4. UN ENREGISTREMENT — CHUTE

```
struct Chute {                // Une goutte :
    uint8_t x;                // sa colonne,
    uint8_t y;                // sa ligne.

    void tomber() { y++; }    // Méthode : descendre d'une ligne (y grandit vers le bas).
};
```

5. UN TABLEAU — PLUIE

```
Chute pluie[COMBIEN];        // Les 3 gouttes.
```

6. UNE VARIABLE — PANIER

```
uint8_t panier = 8;          // La colonne de la gauche du panier (il fait 3 cases).
```

7. UNE VARIABLE — ANCIENPANIER

```
uint8_t ancienPanier = 8;    // Où le panier est dessiné.
```

8. UNE VARIABLE — SCORE

```
uint8_t score = 0;
```

9. UNE VARIABLE — VIES

```
uint8_t vies = 3;
```

10. UNE FONCTION — LANCER()

```
void lancer(uint8_t i) {      // (Re)met la goutte i en haut, à une colonne au hasard.
    pluie[i].x = 1 + hasard() % 18; // hasard() % 18 : 0 à 17 ; + 1 : colonnes 1 à 18.
    pluie[i].y = 2;           // Elle repart de la ligne 2.
}
```

11. UNE FONCTION — ATTRAPE()

```
uint8_t attrape(uint8_t i) {  // Rend 1 si la goutte i tombe DANS le panier.
    return pluie[i].y == SOL && pluie[i].x >= panier && pluie[i].x <= panier + 2;
                                // Elle est à la ligne du panier ET dans ses 3 colonnes :
                                // panier, panier + 1 ou panier + 2.
                                // Exemple : panier = 8 → colonnes 8, 9, 10.
}
```

12. UNE FONCTION — DESSINERPANIER()

```
void dessinerPanier() {      // Déplace le panier à l'écran.
    texte(ancienPanier, SOL, " "); // 3 espaces : efface l'ancien panier,
    texte(panier, SOL, "|#|");    // dessine le nouveau,
    ancienPanier = panier;        // et retient où il est.
}
```

13. UNE FONCTION — DESSINERScore()

```
void dessinerScore() {       // Réécrit le score et les vies en haut.
    nombre(7, 0, score);
    nombre(18, 0, vies, 1);
}
```

14. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    ecran(0);                               // Préparation, écran éteint :
    texte(0, 0, "SCORE:");
    texte(12, 0, "VIES:");
    texte(0, 17, "GAUCHE    DROITE");
    for (uint8_t i = 0; i < COMBIEN; i++) {
        lancer(i);                          //  chaque goutte à une colonne au hasard,
        pluie[i].y = 2 + i * 4;            //  mais à des hauteurs différentes : 2, 6, 10
    }                                       //  (pour qu'elles n'arrivent pas toutes ensemble).
    dessinerPanier();
    dessinerScore();
    ecran(1);                               // On rallume : le jeu commence.

    uint8_t attente = 0;                   // Pour ralentir la pluie.

    while (true) {                         // La boucle du jeu :
        image();

        // Le joueur : à chaque image (le panier est rapide).
        if (bouton(DROITE) && panier < 17) panier++; // 17 + 2 = 19 : le bord droit.
        if (bouton(GAUCHE) && panier > 0) panier--;
        if (panier != ancienPanier) dessinerPanier(); // Ne redessine que s'il a bougé.

        // La pluie : un tour sur huit (elle est lente).
        attente++;
        if (attente < 8) continue; // Pas encore 8 images : on saute la suite du tour.
        attente = 0;

        for (uint8_t i = 0; i < COMBIEN; i++) { // Pour chaque goutte :
            texte(pluie[i].x, pluie[i].y, " "); // efface-la,
            pluie[i].tomber();                  // fais-la descendre d'une ligne.

            if (attrape(i)) {                  // Dans le panier ?
                score++;                      // un point,
                lancer(i);                    // elle repart d'en haut,
                dessinerScore();
            } else if (pluie[i].y >= SOL) {    // Sinon, arrivée en bas, à côté ?
                vies--;                       // une vie en moins,
                lancer(i);                    // elle repart d'en haut,
                dessinerScore();
            }

            texte(pluie[i].x, pluie[i].y, "O"); // Redessine-la.
        }

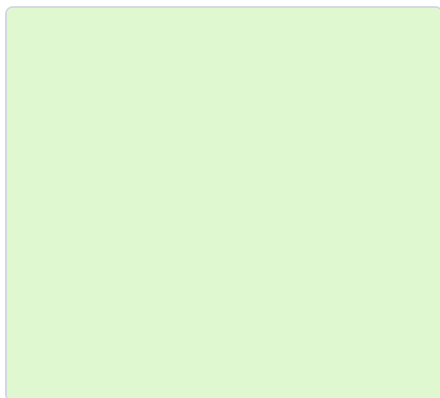
        if (vies == 0) break;                // Plus de vies : on SORT de la boucle du jeu.
    }

    // On arrive ici seulement après le break : la partie est finie.
    texte(6, 8, "PERDU!");
    texte(4, 10, "SCORE:");
    nombre(11, 10, score);

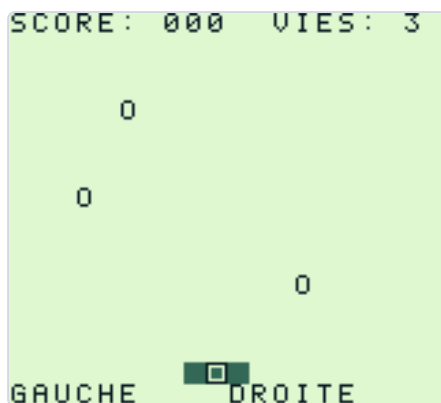
    while (true) {                         // Une dernière boucle pour garder l'écran allumé.
        image();
    }
}
```

3. Ce qui se passe, étape par étape

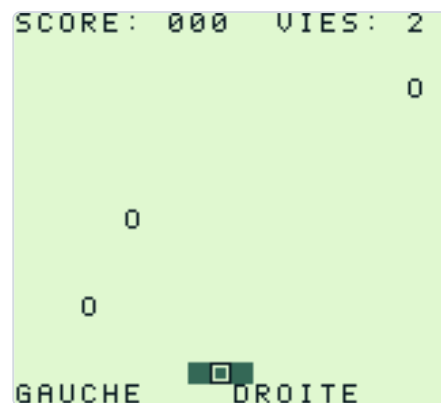
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



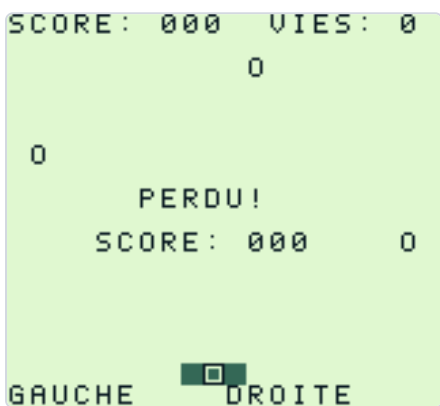
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



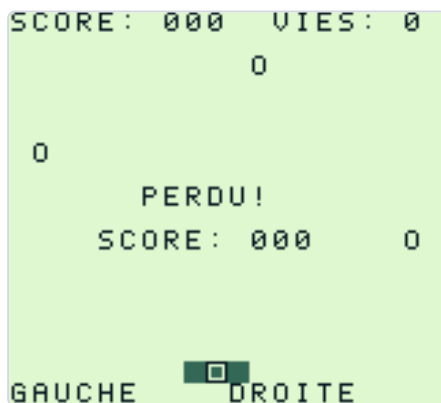
au bout de trois secondes — image 180

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran n'a pas bougé



GAUCHE — l'écran n'a pas bougé

5. Ce qu'on doit voir

Trois O qui tombent, un panier `|#|` que GAUCHE et DROITE déplacent ; sans rien faire, « PERDU! » s'affiche au bout de quelques secondes.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le panier est à l'écran
 - ✓ des objets tombent
 - ✓ GAUCHE déplace le panier (8 puis 7)
 - ✓ chaque objet manqué coûte une vie : on tombe à 0 (après 80 images)
 - ✓ la boucle du jeu se termine par break : PERDU!
-

Ensuite : **94.1. Chapitre 15 — Les quatre nuances : la Game Boy d'origine : ce qui vient** — chapitre 15 : Les quatre nuances : la Game Boy d'origine. · [gameboy3](#) — écrire une cartouche Game Boy en C++